```
public class ExchangeRatesClient
{
    private readonly HttpClient _client;
    public ExchangeRatesClient(HttpClient client)
    {
        _client = client;
        _client.BaseAddress = new Uri("https://example.com/rates/");
    }
}
```

This is functionally equivalent to the approach shown in listing 33.7. It's a matter of taste where you'd rather put the configuration for your `HttpClient`. If you take this approach, you don't need to provide a configuration lambda in `AddHttpClient()`:

```
builder.Services.AddHttpClient<ExchangeRatesClient>();
```

Named clients and typed clients are convenient for managing and encapsulating `HttpClient` configuration, but `IHttpClientFactory` has another advantage we haven't looked at yet: it's easier to extend the `HttpClient` handler pipeline.

33.3 Handling transient HTTP errors with Polly

In this section you'll learn how to handle a common scenario: transient errors when you make calls to a remote service, caused by an error in the remote server or temporary network problems. You'll see how to use `IHttpClientFactory` to handle cross-cutting concerns like this by adding handlers to the `HttpClient` handler pipeline.